

# Tauri: The Lightweight, Flexible Framework for Building Native Desktop Applications



Tauri can create lightweight, fast, and secure applications, while offering cross-platform compatibility and a host of language choices. Let's learn how to build an application with Tauri, and compile it to an executable format.

For ages in software development, GUI development was a mind-numbing task mainly because the GUI frameworks provided very non-intuitive design APIs and lacked flexibility. One reason for the lack of flexibility, in my opinion, was the need for desktop applications to have a uniform look so that users did not get confused or had to learn their way around different applications. While this is true to some extent, those days are now gone. Nowadays, users prefer modern looking GUIs, especially since the advent of web pages. GUI frameworks like QT have evolved to incorporate somewhat different/modern designs. However, they still haven't resolved the issues of non-intuitive design APIs, creating responsive designs, and creating/reusing custom GUI elements.

On the other hand, web pages designed with HTML/CSS have a highly comprehensible style with a minimal learning curve, especially for designers without programming backgrounds. This gave rise to the idea of using web technologies to create GUI applications. One major success resulting from this idea was Electron, a framework for developing cross-platform desktop applications using JavaScript, HTML and CSS. Electron was indeed a fresh breath of air for desktop applications, but it had a major disadvantage — it was essentially a web application running as a desktop application. It used Chromium's rendering library and Node.js as runtime components; so, effectively all Electron applications were web browsers running web applications. This made Electron

applications bulky and less performant compared to lightweight and high performance binaries provided by frameworks like QT.

So, one might ask, "Why not keep the intuitive design or frontend of the application with web technologies and use the OS itself to run the actual working task/backend task?" That's exactly what Tauri provides.

## Why Tauri?

Tauri has several advantages.

*Lightweight and efficient:* Tauri is designed to be lightweight and efficient, making it easier to build and deploy applications that use fewer system resources and have smaller file sizes.

*Flexible and customisable:* It provides a minimal and flexible architecture, allowing developers to create apps that are tailored to their specific needs and requirements.

*Streamlined development experience:* Tauri provides a range of tools and plugins that simplify the development, packaging, and deployment of desktop applications across multiple platforms.

*Native look and feel:* Tauri's use of a web view component enables developers to create desktop applications that have a native look and feel, while still using web technologies for the user interface.

*Active development:* Tauri is a relatively new framework, but it has an active development community that is constantly working to improve and expand its features and capabilities.